Rafael Barbosa

_Desenvolvimento de Software e Operações de TI – DevOps

UNIDADE I



Sumário

1. Cultura DevOps.....3



Cultura DevOps



DevOps – Motivos e histórico

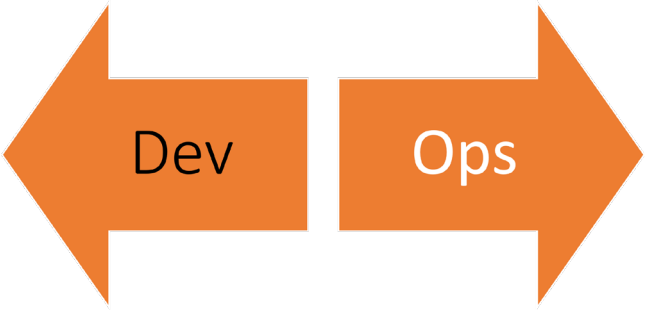
Durante muitos anos, as equipes de desenvolvimento (Dev) e operações (Ops) atuaram de forma separada, com objetivos diferentes e pouca integração. Enquanto os desenvolvedores buscavam entregar novas funcionalidades rapidamente, as equipes de operações priorizavam a estabilidade e a segurança dos ambientes de produção. Essa falta de alinhamento gerava atrasos, retrabalho e dificuldades na entrega de valor ao cliente.

Nesse contexto, surgiu o DevOps, uma abordagem que promove a colaboração entre pessoas, processos e tecnologias para tornar o desenvolvimento e a operação de sistemas mais ágeis, confiáveis e eficientes. A compreensão de sua origem ajuda a entender por que essa cultura

se tornou tão relevante para as organizações modernas.

DevOps é uma cultura que utiliza um conjunto de ferramentas com o objetivo de aperfeiçoar, transformar e promover o aprendizado nos processos de organizações de qualquer porte ou tipo.

Figura – Dev



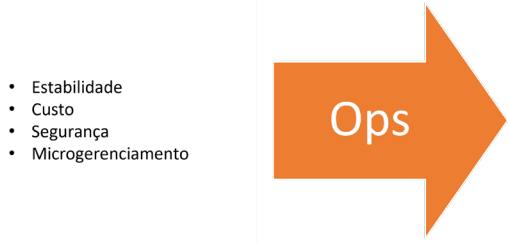
Fonte: elaborada pelo autor.

Figura – Dev



Fonte: elaborada pelo autor.

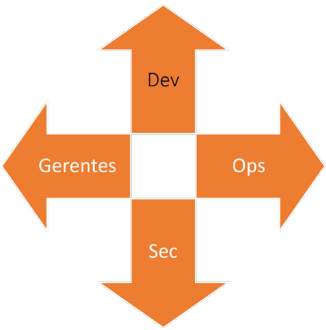
Figura – Ops



Fonte: elaborada pelo autor.

Um dos principais fundamentos do DevOps é eliminar os chamados “silos organizacionais”, que dificultam a comunicação e a colaboração entre equipes. Nesse modelo, profissionais de desenvolvimento, operações, segurança e gestão trabalham de forma integrada, compartilhando responsabilidades e objetivos para aumentar a qualidade, a velocidade e a confiabilidade das entregas.

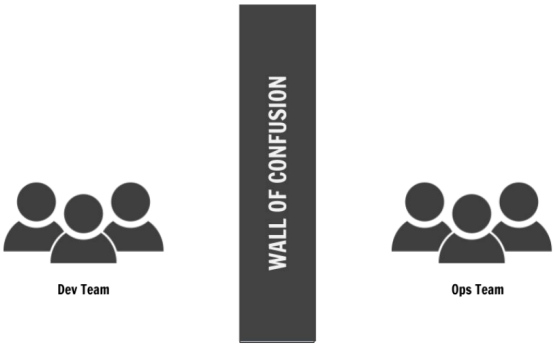
Figura – Integração entre áreas na cultura DevOps



Fonte: elaborada pelo autor.

Antes do surgimento do DevOps, era comum que as equipes de desenvolvimento e operações trabalhassem de forma isolada. Enquanto a equipe de desenvolvimento era responsável por criar e entregar novas funcionalidades, a equipe de operações concentrava-se na estabilidade e manutenção dos ambientes. Essa separação frequentemente gerava falhas de comunicação, conflitos de prioridades e atrasos na entrega de valor ao cliente, fenômeno conhecido como "muro da confusão" (*wall of confusion*).

Figura – Separação entre as equipes de desenvolvimento e operações no modelo tradicional de TI

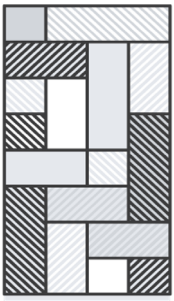


Fonte: adaptada de AWS. Disponível em: https://aws.amazon.com/devops/what-is-devops/.

2001 – Predominância das aplicações monolíticas

Em 2001, predominavam aplicações monolíticas, nas quais todas as funcionalidades eram concentradas em um único sistema. Essa característica influenciava também a estrutura das equipes, que atuavam de forma centralizada e com pouca especialização.

Figura – Representação de uma aplicação monolítica e de equipes centralizadas



aplicação monolítica

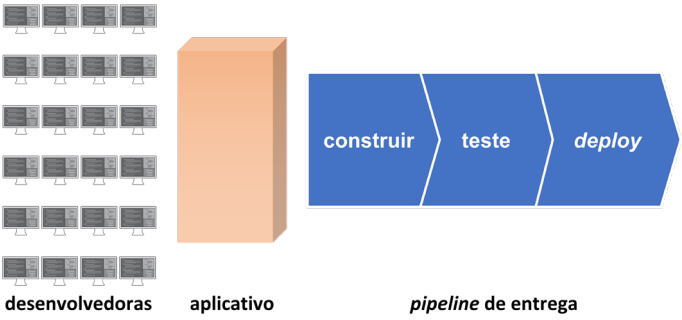
+

equipes monolíticas

Fonte: elaborada pelo autor.

Ciclo de vida de desenvolvimento de uma aplicação monolítica

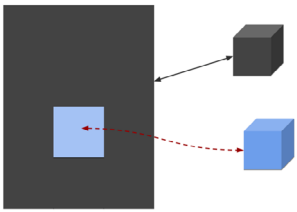
Figura – Ciclo de vida de desenvolvimento de uma aplicação monolítica



Fonte: elaborada pelo autor.

Em aplicações monolíticas, todas as funcionalidades do sistema são desenvolvidas, testadas e implantadas como uma única unidade. Por esse motivo, qualquer alteração exige a execução de todo o ciclo de entrega, o que pode aumentar o tempo de implantação e a dependência entre equipes.

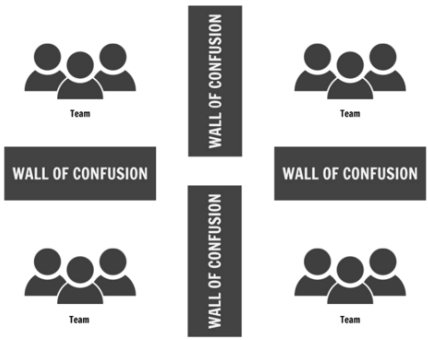
Figura – Microsserviços



Fonte: elaborada pelo autor.

Antes da adoção da cultura DevOps, era comum que as equipes de desenvolvimento, testes, operações e demais áreas de TI atuassem de forma isolada. Essa separação criava barreiras de comunicação, desalinhamento de objetivos e transferência de responsabilidades entre equipes, situação conhecida como *wall of confusion* (muro da confusão). O DevOps surgiu justamente para reduzir esses silos organizacionais e promover a colaboração entre todos os envolvidos no ciclo de desenvolvimento e operação dos sistemas.

Figura – *Wall of confusion*: barreiras de comunicação entre equipes de TI



Fonte: elaborada pelo autor.

Microsserviços: considerações

- Centralização de *log*
- Monitoramento das aplicações

- Monitoramento de rede entre os microsserviços
- Aumento da complexidade dos recursos alocados: servidores, balanceadores de carga, rede etc.
- Boa comunicação entre equipes
- Um problema em um serviço pode gerar uma reação em cadeia
- *Circuit breaker*

Lei de Conway

> "Organizações que criam sistemas estão restritas a produzir seus projetos no quais são cópias das estrutura de comunicação dessa organizações."
>
> (CONWAY, Melvin E. How Do Committees Invent? **Datamation**, v. 14, n. 4, p. 28-31, 1968.)

O DevOps surgiu como uma resposta aos desafios enfrentados pelas organizações para entregar *software* com mais rapidez e qualidade. Ao longo das últimas décadas, práticas, métodos e tecnologias contribuíram para essa evolução.

A linha do tempo apresentada a seguir destaca alguns marcos importantes que influenciaram o

desenvolvimento da cultura DevOps, culminando na consolidação dos conceitos apresentados em obras como “The Phoenix Project” e “The Three Ways”.

“The Three Ways”

Gene Kim

1913	1948	1988	1990	1996	2001	2006	2008	2009	2010	2013

Culture Automation Measurement Sharing

Em 2010, Damon Edwards e John Willis propuseram o modelo CAMS, um conjunto de princípios que ajudou a consolidar a cultura DevOps.

CAMS é um acrônimo para:

- ***Culture* (Cultura):** colaboração entre desenvolvimento, operações e demais áreas.
- ***Automation* (Automação):** redução de atividades manuais por meio de ferramentas e processos automatizados.
- ***Measurement* (Medição):** acompanhamento de métricas para avaliar desempenho e orientar melhorias.

- ***Sharing*** (**Compartilhamento**): disseminação de conhecimento, experiências e boas práticas entre as equipes.

***C**ulture **A**utomation **M**easurement **S**haring*

Damon Edwards e John Willis

_DevopsDays Ghent

Em 2009, Patrick Debois organizou o DevOpsDays Ghent, na Bélgica. O evento reuniu profissionais de desenvolvimento e operações para discutir formas de melhorar a colaboração entre essas áreas, que tradicionalmente trabalhavam de forma isolada.

O encontro é considerado um marco na história do DevOps, pois ajudou a popularizar o termo e a disseminar práticas voltadas à integração entre equipes, automação de processos e entrega contínua de *software*.

DevopsDays Ghent

Patrick Debois

_Agile Infrastructure & Operation

Em 2008, Patrick Debois e Andrew Shafer promoveram discussões sobre como aplicar princípios ágeis às equipes de infraestrutura e operações.

Essa aproximação entre desenvolvimento e operações é considerada um dos principais antecedentes do movimento DevOps.

Agile Infrastructure & Operation

Patrick Debois e Andrew Shafer

http://www.jedi.be/presentations/agile-infrastructure-agile-2008.pdf

http://www.jedi.be/presentations/IEEE-Agile-Infrastructure.pdf

1913	1948	1988	1990	1996	2001	2006	***2008***	2009	2010	2013

_The Deployment Production Line

Em 2006, Dan North, Jez Humble e Chris Read apresentaram o conceito de Deployment Production Line, propondo que a entrega de *software* fosse tratada como um fluxo contínuo e automatizado, semelhante a uma linha de produção.

A ideia defendia que cada alteração no código passasse por etapas padronizadas de compilação, testes e validação antes de chegar ao ambiente de produção. Esse conceito contribuiu para a evolução de práticas que posteriormente deram origem à Integração Contínua (CI) e à Entrega Contínua (CD), pilares fundamentais do DevOps.

The Deployment Production Line

Dan North, Jez Humble e Chris Read

https://continuousdelivery.com/wp-content/uploads/2011/04/deployment_production_line.pdf

1913	1948	1988	1990	1996	2001	***2006***	2008	2009	2010	2013

Agile Manifesto, 2001

Indivíduos e interações mais que processos e ferramentas.

***Software* em funcionamento** mais que documentação abrangente.

Colaboração com o cliente mais que negociação de contratos.

Responder a mudanças mais que seguir um plano.

Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda.

https://agilemanifesto.org/iso/ptbr/manifesto.html

_*Lean thinking*

Em 1996, James P. Womack e Daniel T. Jones consolidaram os princípios do *lean thinking*, abordagem focada na geração de valor para o cliente por meio da eliminação de desperdícios e da melhoria contínua dos processos.

Conceitos como redução de gargalos, otimização do fluxo de trabalho e busca contínua por eficiência influenciaram diretamente práticas que mais tarde seriam incorporadas ao DevOps.

_*The Machine that Changed the World* (1990)

Em 1990, James P. Womack, Daniel Jones e Daniel Roos publicaram “The Machine that Changed the World”, obra que apresentou ao Ocidente os princípios do Sistema Toyota de Produção.

O livro popularizou o conceito de produção enxuta (*lean manufacturing*), baseado na eliminação de desperdícios, melhoria contínua e otimização do fluxo de valor.

Esses princípios influenciaram diversas abordagens de gestão e desenvolvimento de *software*, tornando-se uma das bases conceituais que mais tarde contribuíram para o surgimento do DevOps.

1913	1948	1988	***1990***	1996	2001	2006	2008	2009	2010	2013

_ *Triumph of the Lean Production System*

Em 1988, John Krafcik publicou o artigo "Triumph of the Lean Production System", no qual apresentou o termo ***lean*** para descrever o modelo de produção adotado pela Toyota.

O artigo destacou como a eliminação de desperdícios, a melhoria contínua e o foco na geração de valor permitiam alcançar melhores resultados em qualidade, produtividade e eficiência.

Esses princípios influenciaram diversas áreas além da manufatura, incluindo o desenvolvimento de *software*, tornando-se uma das bases conceituais do DevOps.

Triumph of the Lean Production System

John Krafcik

https://search.proquest.com/openview/a6938b5d5125c0061cc9881c8014c9bd/1?pq-origsite=gscholar&cbl=26142

1913	1948	***1988***	1990	1996	2001	2006	2008	2009	2010	2013

Toyota Production System

Desenvolvido por Taiichi Ohno e Eiji Toyoda, o Toyota Production System (TPS) introduziu princípios voltados à eliminação de desperdícios, melhoria contínua da qualidade e otimização do fluxo de trabalho.

Conceitos como *Just in Time*, Kanban, redução de estoques e identificação de gargalos tornaram-se referência mundial e influenciaram posteriormente abordagens como *lean manufacturing*, *lean thinking* e, mais tarde, DevOps.

Toyota Production System

Taiichi Ohno e Eiji Toyoda

1913	***1948***	1988	1990	1996	2001	2006	2008	2009	2010	2013

Assembly Line – Henry Ford

Em 1913, Henry Ford introduziu a linha de montagem móvel na produção de automóveis. Esse modelo permitiu dividir o trabalho em etapas padronizadas, reduzindo o tempo de produção e aumentando a eficiência operacional.

A padronização dos processos, a redução de desperdícios e a busca por melhoria contínua influenciaram modelos de gestão que, décadas mais tarde, contribuíram para o surgimento dos princípios Lean e, consequentemente, do DevOps.

Linha de montagem

Henry Ford

1913	1948	1988	1990	1996	2001	2006	2008	2009	2010	2013

DevOps

DevOps – Certificações

Figura – Certificações



Fonte: elaborada pelo autor.

DevOps – É profissão?

DevOps é uma abordagem cultural e organizacional que busca acelerar a entrega de valor por meio da adoção de práticas inspiradas em métodos ágeis (*agile*) e nos princípios *lean*.

Seu foco está na colaboração entre as equipes de desenvolvimento e operações, promovendo maior integração, compartilhamento de responsabilidades e melhoria contínua dos processos.

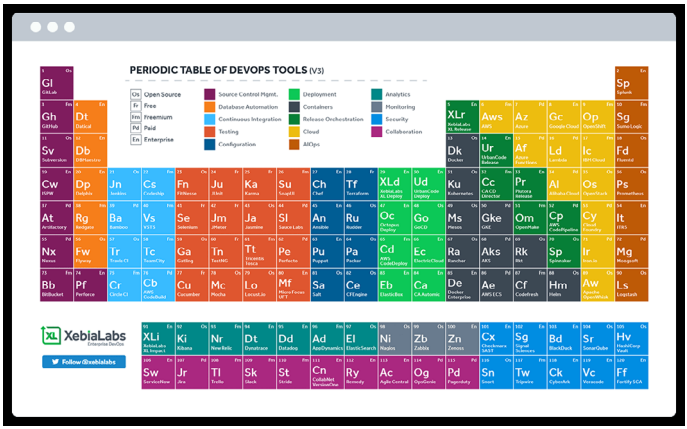
Para apoiar essa transformação, são utilizadas tecnologias e ferramentas de automação que tornam a infraestrutura mais programável, escalável e alinhada ao ciclo de vida das aplicações.

Ferramentas DevOps:

- SCM
- *Continuous Integration*
- *Deployment*
- *Cloud* / IaaS / PaaS
- BI / *Monitoring*
- *Database Management*

- *Repository Management*
- *Configuration / Provisioning*
- *Release Management*
- *Logging*
- *Build*
- *Testing*
- *Containeirzation*
- *Collaboration*
- *Security*

Figura – Exemplo da diversidade de ferramentas associadas ao ecossistema DevOps



Fonte: XebiaLabs. Periodic Table of DevOps Tools (v3). Disponível em: https://xebialabs.com. Acesso em: jul. 2026.

O professional que gerencia as ferramentas descritas anteriormente é o *Site Reliability Engineer (*SER). Vale ressaltar, porém, que DevOps é uma ideia de cultura; assim, sua implementação é responsabilidade de todos na empresa.

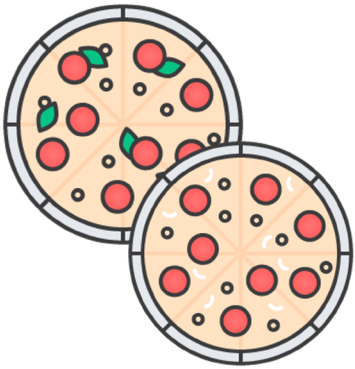
DevOps na Amazon

Two-pizza team (equipe de duas pizzas)

Modelo organizacional popularizado pela Amazon, no qual as equipes são pequenas o suficiente para serem alimentadas com duas pizzas. O objetivo é aumentar a autonomia, a comunicação e a velocidade na tomada de decisões.

- Propriedade total
- Responsabilidade total
- Incentivos alinhados
- Colaboração ente desenvolvimento e operações

Figura – Representação do conceito de *Two-Pizza Team*



Fonte: elaborada pelo autor.

Como funcionam as equipes de duas pizzas?

Como funcionam as equipes de duas pizzas?

Na Amazon, as equipes de duas pizzas são organizadas em torno de um produto ou serviço específico e possuem autonomia para tomar decisões e conduzir seu trabalho.

Essas equipes são responsáveis por todo o ciclo de vida do serviço, incluindo:

- planejamento e definição do *roadmap*;
- desenvolvimento e evolução da solução;
- operação, suporte e atendimento aos usuários.

Esse modelo é frequentemente resumido pela expressão:

"Você constrói, você opera" (*You build it, you run it*).

Embora tenham autonomia, as equipes fazem parte de uma estrutura organizacional maior, como Amazon.com, AWS e Amazon Prime, compartilhando diretrizes, padrões e objetivos corporativos.

Quem faz o controle de qualidade?



Controle de qualidade nas equipes de duas pizzas

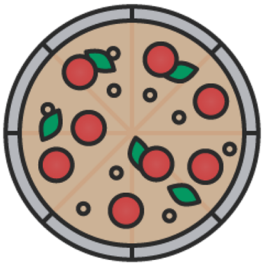
A qualidade é responsabilidade de toda a equipe.

Os integrantes participam da definição, implementação e validação dos requisitos.

Testes automatizados e revisões de código são incorporados ao processo de desenvolvimento.

A equipe é responsável pela qualidade durante todo o ciclo de vida do produto.

Quem está de plantão?



O plantão não é uma pessoa específica, mas sim a capacidade da equipe de responder aos eventos e demandas operacionais. Em um ambiente saudável, as atividades são distribuídas entre os membros do time, garantindo cobertura, continuidade e redução da dependência de indivíduos. Assim, o "plantonista" é representado pela própria equipe, preparada para atuar de forma colaborativa diante das necessidades do negócio.

O que o Ops faz?



O que o Ops faz?

- Implantação e atualização de aplicações.
- Monitoramento e observabilidade.
- Resposta a incidentes.
- Gestão da infraestrutura.
- Automação de processos operacionais.

E as operações, a infraestrutura e a qualidade?

Embora as equipes tenham autonomia para desenvolver e entregar seus produtos, elas não trabalham isoladas. A organização conta com equipes especializadas que fornecem suporte, ferramentas, padrões e conhecimento para toda a empresa.

Essas equipes especializadas incluem:

- **Engenheiros de *Software* (SDEs):** foco no desenvolvimento de aplicações e funcionalidades.
- **Gerentes de Produto (PMs):** responsáveis por definir a visão, os objetivos e o direcionamento do produto.
- **Gerentes Técnicos de Programa (TPMs):** atuam na coordenação de iniciativas e na remoção de impedimentos.
- **Engenheiros de Sistemas (SEs):** responsáveis por infraestrutura, plataformas e ferramentas compartilhadas.
- **Engenheiros de Testes (SDETs):** apoiam a qualidade por meio de automação de testes e disseminação de boas práticas.

Na maioria das equipes de duas pizzas, os papéis de desenvolvimento e produto estão presentes diretamente na equipe, enquanto especialistas de infraestrutura, testes e outras áreas podem atuar de forma compartilhada entre diferentes equipes.

Autonomia com responsabilidade

À primeira vista, conceder mais autonomia às equipes pode parecer perda de controle. No entanto, no modelo DevOps, essa autonomia é acompanhada por práticas, padrões e mecanismos de governança que garantem alinhamento, qualidade e segurança nas entregas.

As equipes são capacitadas e apoiadas por meio de:

- programas estruturados de integração e treinamento, garantindo o domínio das ferramentas e processos utilizados pela organização;
- padrões e boas práticas corporativas, construídos a partir da experiência acumulada pela empresa e aplicados em larga escala;
- acompanhamento periódico de métricas técnicas e de negócio, permitindo avaliar resultados e identificar oportunidades de melhoria;
- compartilhamento contínuo de conhecimento, incluindo novas ferramentas, tecnologias e práticas apresentadas por especialistas internos;

- análise e divulgação das lições aprendidas após incidentes, promovendo aprendizado coletivo e evitando a repetição de problemas.

Para aumentar a autonomia e a velocidade de entrega das equipes:

- **Autoatendimento:** reduz dependências operacionais.
- **Independente de tecnologia:** oferece flexibilidade na adoção de ferramentas e plataformas.
- **Incentivo às melhores práticas:** promove qualidade, segurança e padronização.
- **Serviços de propósito único:** simplificam manutenção e evolução dos sistemas.

Como parte da evolução dos processos de entrega de *software*, algumas práticas tornaram-se fundamentais para garantir implantações mais rápidas, seguras e previsíveis:

- **serviço de implantação:** automatiza a publicação de novas versões dos sistemas.
- **implantações sem tempo de inatividade:** permitem atualizar aplicações sem interromper o serviço para os usuários;

- **verificação de saúde:** monitora aplicações e infraestrutura para identificar rapidamente falhas após a implantação;
- **artefatos e reversões versionados:** garantem rastreabilidade das versões implantadas e possibilitam retorno rápido a uma versão estável quando necessário.

Os resultados obtidos com a adoção de práticas ágeis e da automação foram significativos, permitindo que as equipes desenvolvessem e entregassem soluções com mais rapidez. No entanto, estudos posteriores mostraram que ainda existiam oportunidades para melhorar o fluxo de trabalho e aumentar a eficiência dos processos.

Em 2009, um estudo identificou que muitas equipes ainda tinham sua produtividade reduzida por processos manuais e gargalos no fluxo de trabalho.

Ações e transições automatizadas:

do *check-in* à produção

Benefícios do desenvolvimento:

- mais rápido;
- mais seguro;
- consistente e padronizado;
- visualização do processo.

As três maneiras (*The three ways)*

DevOps – Três maneiras: fluxo

A primeira maneira: fluxo

O primeiro princípio do DevOps é promover um fluxo contínuo e eficiente de trabalho, desde o desenvolvimento até as operações e, por fim, a entrega de valor ao cliente.

Ao melhorar o fluxo de valor, reduzimos o tempo de espera entre a solicitação de uma demanda e sua disponibilização para uso. Isso permite atender mais rapidamente às necessidades internas e dos clientes, além de diminuir o tempo necessário para implementar mudanças em produção.

Com processos mais ágeis e previsíveis, a equipe ganha eficiência, melhora a qualidade das entregas e amplia sua capacidade de inovação,

contribuindo também para a competitividade da organização.

Para acelerar o fluxo de valor, ou seja, reduzir o tempo entre a solicitação de uma demanda e a entrega de valor ao cliente, algumas práticas são fundamentais:

- **tornar o trabalho visível:** utilizar ferramentas visuais, como quadros Kanban, para acompanhar o andamento das atividades e identificar prioridades;
- **limitar o trabalho em andamento (WIP):** restringir a quantidade de tarefas executadas simultaneamente para aumentar o foco e reduzir atrasos;
- **reduzir o tamanho dos lotes de trabalho:** dividir grandes entregas em partes menores, facilitando validações e correções ao longo do processo;
- **reduzir o número de transferências:** minimizar a passagem de atividades entre equipes ou pessoas, reduzindo perdas de informação e tempo de espera;

- **identificar e melhorar as restrições:** reconhecer gargalos que limitam a capacidade de entrega e atuar para eliminá-los ou mitigá-los;
- **eliminar atritos e desperdícios:** remover atividades que não agregam valor ao cliente, como retrabalho, filas excessivas e processos desnecessários.

_Tornar o trabalho visível

Em equipes de tecnologia, grande parte do trabalho não é facilmente observada, pois envolve atividades como análise, desenvolvimento, testes e correção de problemas. Isso pode dificultar a compreensão do andamento das demandas e da capacidade da equipe.

Para tornar o trabalho mais transparente, podem ser utilizados quadros visuais, como o Kanban, que organizam as atividades de acordo com cada etapa do processo. Dessa forma, todos conseguem visualizar quais tarefas estão em andamento, quais aguardam execução e quais já foram concluídas.

Com maior visibilidade, torna-se mais fácil identificar prioridades, acompanhar prazos, detectar gargalos e direcionar esforços para as atividades que geram mais valor para o negócio.

Limitar o WIP (*Work in Progress*)

Na área de TI, a rotina costuma ser dinâmica, com demandas provenientes de diferentes áreas da organização e recebidas por vários canais de comunicação, como e-mail, Teams, telefone e sistemas corporativos.

Estudos sobre produtividade indicam que executar várias tarefas simultaneamente pode aumentar o tempo necessário para concluí-las e impactar a qualidade das entregas. Embora pareça mais produtivo trabalhar em diversas demandas ao mesmo tempo, a constante alternância entre atividades exige esforço adicional para retomar informações, reorganizar o raciocínio e recuperar o contexto de cada tarefa.

Uma forma de controlar o trabalho em progresso (WIP) é utilizar um quadro Kanban, ferramenta visual que organiza as atividades em colunas que representam as etapas do fluxo de trabalho.

Nesse método, define-se um limite para a quantidade de cartões que podem permanecer simultaneamente em uma coluna, como “Em andamento”. Dessa forma, a equipe prioriza a conclusão das atividades já iniciadas antes de começar novas demandas.

Além disso, a limitação de WIP contribui para a identificação de gargalos e impedimentos que dificultam o avanço das atividades.

Reduzir o tamanho do lote de trabalho

No *Lean*, uma abordagem voltada à melhoria contínua e à eliminação de desperdícios, uma das principais lições é que reduzir o tamanho dos lotes permite entregar valor mais rapidamente, identificar problemas mais cedo e melhorar a qualidade.

O cenário ideal é o fluxo unitário, no qual cada item percorre o processo individualmente, sem aguardar a conclusão de um grande conjunto de atividades.

Quando os lotes são grandes, aumenta a quantidade de trabalho em andamento (WIP – *Work in Progress*), o que gera mais filas, mais

tempo de espera e maior demora para que o valor chegue ao cliente.

Em DevOps, a redução do tamanho dos lotes contribui para acelerar o fluxo, reduzir riscos e obter *feedback* mais rápido sobre as mudanças realizadas.

No exemplo da figura a seguir, cada carta passa pelas etapas de dobrar, inserir no envelope, selar e carimbar. Quando todas as cartas são processadas em grandes lotes, o primeiro item só fica pronto após a conclusão de todas as etapas. Já no fluxo unitário, cada carta percorre o processo individualmente, permitindo que o primeiro produto seja entregue mais rapidamente e reduzindo o tempo de espera entre as atividades.

F = Dobrar a carta

I = Inserir a carta no envelope

Se = Selar o envelope

St = Carimbar o envelope

Legenda da figura:

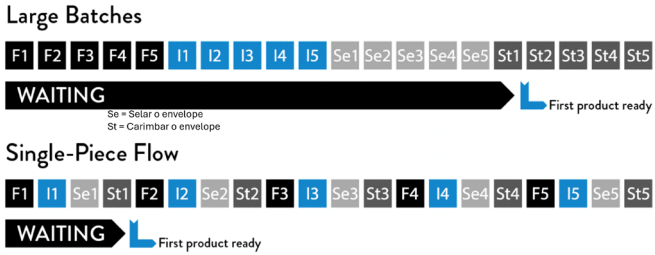
Large Batches = Grandes lotes

Single-Piece Flow = Fluxo unitário

Waiting = Espera

First Product Ready = Prim

Figura – Reduzir o tamanho do lote de trabalho



Fonte: elaborada pelo autor.

F = Folding (dobrando a carta)

I = Inserting (adicionando a carta no envelope)

Se = Sealing (fechando e colando o envelope)

Ts = Stamping (carimbando o envelope)

Reduzir o número de transferências

Um dos fatores que mais contribuem para o aumento do tempo de entrega é a quantidade de transferências de atividades entre equipes e departamentos ao longo do fluxo de valor. Cada transferência pode gerar filas, perda de contexto, retrabalho e atrasos.
Um exemplo comum é a chamada última milha da entrega, que corresponde às etapas finais necessárias para disponibilizar uma solução em produção. Em organizações tradicionais, essas atividades costumam depender de diversas equipes especializadas, aumentando o tempo de espera e a complexidade do processo.

Entre as atividades normalmente envolvidas nessa etapa estão:

- realização de testes funcionais;
- execução de testes de integração;
- provisionamento de ambientes;
- configuração e gerenciamento da infraestrutura necessária para a aplicação, como servidores, armazenamento, redes e balanceadores de carga.

Sempre que realizamos transferências, corremos o risco de perder informações devido a problemas de comunicação.

Sempre que ocorrem transferências, criamos filas (desperdício) entre pessoas ou departamentos. Muitas vezes, é necessário escalar a demanda para que os gestores a priorizem.

Existem algumas formas de reduzir o número de transferências:

- automação de processos (todo o processo ou partes dele);
- reorganização de equipes, para que tenham autonomia e não dependam de outros times.

Identificar e mitigar as restrições

Restrições são os gargalos que limitam a velocidade do fluxo de valor.

Para reduzir o tempo necessário para entregar valor ao cliente e aumentar a capacidade de entrega do sistema, precisamos identificar continuamente essas restrições e atuar para eliminá-las ou reduzir seus impactos.

Segundo Eliyahu Goldratt:

> "Em qualquer fluxo de valor existe um fator que limita seu desempenho. Melhorias realizadas fora desse ponto têm impacto limitado nos resultados do sistema como um todo."

> "Em qualquer fluxo de valor existe um fator que limita seu desempenho. Melhorias realizadas fora desse ponto têm impacto limitado nos resultados do sistema como um todo."

> Por isso, a identificação e o tratamento dos gargalos devem ser priorizados para aumentar a eficiência do fluxo de trabalho.

(GOLDRATT, Eliyahu M.; COX, Jeff. **A meta:** um processo de melhoria contínua. 2. ed. São Paulo: Nobel, 2014.)

Em uma transformação DevOps, à medida que gargalos são eliminados, a principal restrição do fluxo costuma migrar para outras etapas do processo. Identificar onde o trabalho está ficando acumulado permite direcionar os esforços de melhoria para o ponto de maior impacto.

Exemplos de restrições comuns:

- **criação de ambientes:** quando a disponibilização de ambientes para

desenvolvimento, testes ou produção é lenta e depende de atividades manuais;

- **implementação de código:** quando a capacidade da equipe de desenvolvimento não acompanha a demanda por novas funcionalidades ou correções;
- **execução de testes:** quando os testes são demorados, manuais ou insuficientemente automatizados, atrasando a entrega;
- **arquitetura altamente acoplada:** quando alterações em um componente exigem mudanças em vários outros sistemas, tornando a evolução mais lenta e arriscada.

Passos para melhorar o fluxo de valor:

- Identificar a restrição do sistema
- Localizar o principal gargalo que está limitando a entrega de valor.
- Explorar a restrição
- Utilizar a capacidade existente da melhor forma possível, reduzindo desperdícios e interrupções.
- Subordinar o restante do sistema à restrição

- Ajustar as demais atividades para apoiar o gargalo, evitando gerar mais trabalho do que ele consegue processar.
- Elevar a restrição
- Aumentar a capacidade do gargalo por meio de melhorias, automação, treinamento ou recursos adicionais.
- Recomeçar o ciclo de melhoria
- Quando uma restrição é eliminada, identificar o próximo gargalo e repetir o processo.

Eliminar atritos e desperdícios no fluxo de valor

No *Lean*, uma abordagem de gestão focada em maximizar a entrega de valor ao cliente e eliminar desperdícios. Desperdício é qualquer atividade que não agrega valor ou causa atraso na entrega.

No desenvolvimento de *software*, alguns exemplos de desperdício são:

- **trabalho parcialmente concluído:** funcionalidades iniciadas, mas ainda não entregues;

- **processos desnecessários:** aprovações, controles ou etapas que não agregam valor ao resultado final;
- **troca constante de tarefas:** alternância frequente de contexto, reduzindo a produtividade;
- **transferências entre equipes:** repasses excessivos de atividades ou informações;
- **tempo de espera:** filas para aprovação, testes, implantação ou análise;
- **defeitos:** erros que exigem correção e retrabalho;
- **trabalho manual ou não padronizado:** atividades repetitivas que poderiam ser automatizadas ou executadas de forma consistente.

DevOps – Três maneiras: *feedback*

A segunda maneira – *feedback*

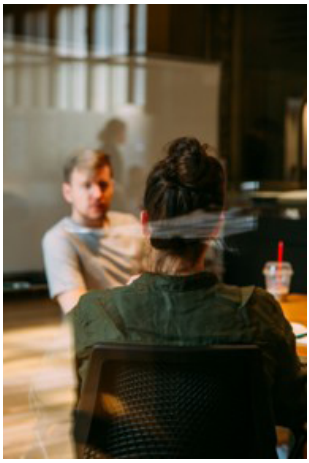
Falhas são inevitáveis em sistemas complexos. Por isso, devemos criar mecanismos que permitam identificar problemas rapidamente, compartilhar informações entre as equipes e agir antes que pequenos erros se transformem em falhas graves.

Isso é alcançado por meio de:

- *feedback* rápido e contínuo;
- colaboração entre áreas;
- qualidade incorporada ao processo;
- resolução de problemas na origem;
- aprendizado compartilhado e melhoria contínua.

Resolver os problemas em conjunto para construir novos conhecimentos

Figura – Resolução em conjunto



Fonte: https://unsplash.com/pt-br/fotografias/pessoa-sentada-em-uma-cadeira-na-frente-de-um-homem-rRWiVQzLm7k.

Benefícios do *Andon Cord* (Farol):

- fornece *feedback* rápido sobre falhas e desvios;
- acelera o diagnóstico e a solução dos problemas;
- direciona a atenção da equipe para a resolução da causa raiz;
- reduz a recorrência de falhas;
- gera aprendizado compartilhado para toda a organização;
- preserva informações importantes para análise e melhoria contínua.

Manter o controle de qualidade próximo da origem do trabalho

Em vez de depender de inspeções e aprovações realizadas apenas nas etapas finais do processo, precisamos que todas as pessoas envolvidas no fluxo de valor atuem continuamente na identificação e na resolução de problemas em sua área de responsabilidade.

Ao fazer isso, transferimos a responsabilidade pela qualidade, segurança e tomada de decisão

para o ponto em que o trabalho é executado, permitindo respostas mais rápidas, maior autonomia e redução da dependência de aprovações hierárquicas distantes.

Formas de mitigação:

- aplicar *peer reviews (revisão em pares)* em mudanças;
- automação de todo o trabalho possível;
- permitir a realização de testes sob demanda.

Otimizar o trabalho pensando na próxima etapa do fluxo

De acordo com o *Lean*, o cliente mais importante é quem recebe nosso trabalho assim que o concluímos. Para otimizar nosso trabalho, é necessário compreender os problemas, a fim de identificar e eliminar obstáculos que possam comprometer o fluxo rápido e sem atritos.

DevOps – Três maneiras: aprendizado contínuo e experimentação

A terceira maneira – aprendizado contínuo e experimentação

A terceira maneira possibilita a criação de uma cultura de alta confiança que favorece uma abordagem dinâmica, disciplinada e científica para experimentar e assumir riscos, facilitando a aprendizagem organizacional.

Reduzindo continuamente os ciclos de *feedback*, criamos sistemas de trabalho cada vez mais seguros, melhorando a capacidade de assumir riscos e de realizar experimentos que nos ajudam a aprender mais raapidamente.

Como parte da terceira maneira, também projetamos sistemas de trabalho para multiplicar os efeitos de novos conhecimentos e transformar as descobertas locais em melhorias globais.

Institucionalizar a melhoria do trabalho diário

Melhoramos nosso trabalho diário quando reservamos tempo para reduzir o débito técnico,

corrigir defeitos e refatorar partes problemáticas do nosso código e do ambiente.

Podemos fazer isso:

- reservando um período de tempo em nosso ciclo de desenvolvimento;
- agendando *Kaizen Blitzes*.

Transformar descobertas locais em melhorias globais

Quando um aprendizado é obtido por uma equipe, devemos criar mecanismos para que esse conhecimento também seja compartilhado com o restante da empresa. Assim, quando alguém enfrentar uma situação semelhante, poderá aproveitar o conhecimento acumulado e colaborativo da organização.

Fazemos isso da seguinte forma:

- tornando os relatórios de *blameless post-mortem* acessíveis para todos;
- criando um repositório compartilhado de código, bibliotecas e arquivos de configuração.

Esses mecanismos ajudam a transformer o conhecimento individual em artefatos dos quais toda a organização pode tirar proveito.

Adicionar testes de resiliência ao trabalho diário

Uma prática para aumentar a performance de um ambiente ou de uma equipe é aprimorar constantemente o trabalho diário.

Fazemos isso por meio das seguintes ações:

- buscando reduzir o *deployment lead time*;
- aumentando a cobertura de testes no código;
- reduzindo tempo de execução de testes;
- alterando a arquitetura da aplicação visando aprimorar a confiabilidade do serviço;
- executando *game days* em que são simuladas falhas em larga escala (desligar um *data center*);
- injetando falhas na infraestrutura para avaliar seu comportamento e identificar orpotunidades de melhoria da resiliência.

Simian Army (Netflix): conjunto de ferramentas criado para testar continuamente a resiliência dos sistemas por meio de experimentos controlados em produção. A iniciativa foi pioneira na prática de *Chaos Engineering*.

Três exemplos de ferramentas do Simian Army:

- **Chaos Monkey**: desliga aleatoriamente instâncias e serviços em produção para verificar se a aplicação consegue se recuperar automaticamente.
- **Latency Monkey**: introduz atrasos artificiais na comunicação entre serviços para testar a tolerância do sistema a problemas de desempenho.
- **Chaos Gorilla**: simula a indisponibilidade de uma zona inteira de disponibilidade (Availability Zone), avaliando a capacidade de recuperação em cenários de falhas de maior impacto.

Líderes devem reforçar uma cultura de aprendizado

Jim Womack, autor do livro «Gemba Walks», descreve a relação complementar e de respeito mútuo que deve haver entre líderes e seus colaboradores.

"Este tipo de relacionamento é necessário porque nenhum deles pode resolver problemas sozinho. Enquanto líderes não estão perto o suficiente do trabalho para terem condições de resolver problemas, seus liderados não possuem o contexto de toda a organização ou a autoridade para a tomada de decisões fora do seu escopo de trabalho."

Líderes devem elevar o valor do aprendizado e da disciplina de resolução de problemas.

(WOMACK, James P. **Gemba Walks**. Cambridge, MA: Lean Enterprise Institute, 2011. 348p.)

Blameless post-mortem

A teoria da maçã podre pressupõe que um sistema é inerentemente seguro, exceto para o indivíduo problemático.

Uma empresa sem culpa (*blameless*) parte do princípio de que os sistemas NÃO são inerentemente seguros e que as pessoas fazem o melhor que podem para mantê-los funcionando.

Dica 1 – 3 Rs

REGREAT (ARREPENDIMENTO)

- Comece reconhecendo o problema e pedindo desculpas pelo que aconteceu.
- Demonstre empatia tanto pelos clientes afetados pela interrupção quanto pelas pessoas envolvidas no incidente.

REASON (RAZÃO)

- Isso deve abranger todo o processo, desde a detecção inicial de incidentes até sua resolução.
- Quanto mais detalhes você puder fornecer, melhor.
- Não faz sentido esconder fatos.

REMEDY (REMÉDIO)

- Certifique-se de que as ações de remediação atendam aos critérios SMART (*Specific Measurable Attainable Realistic Time Based*).

- Tudo bem se você ainda não conhece a solução, mas deve se comprometer a continuar a investigação.
- Comunique às equipes internas e aos clientes que o problema esta´ sendo tratado e que há um plano para evitar novas interrupções.

Dica 2 – Mudar o foco da interrupção

Você está focando no Tempo Médio de Resolução (MTTR) ou no Tempo Médio Entre Falhas (MTBF)? Provavelmente, você está pensando demais em acrônimos e não o suficiente no panorama geral. Mudar a forma como você percebe a falha é essencial se você deseja melhorar continuamente ao longo do tempo.

> "Nós nos concentramos em aprender com os incidentes, melhorar o ambiente em uma organização, o ambiente em que as pessoas estão trabalhando. O ambiente que contribui para o modo como as pessoas que trabalham nele se comportam tanto durante o trabalho normal quanto durante situações estressantes."

MEYER, Mathias. **Success and failure:** two sides of the same coin. Apresentação na HashiConf, HashiCorp, 2016.

Disponível em: YouTube/HashiCorp.

Dica 3 – Parar de procurar a causa raiz

A maioria das interrupções de produção, senão todas, resulta de diversos fatores. As decisões tomadas meses ou anos antes podem, eventualmente, contribuir para um problema que ninguém seria capaz de prever naquele momento. Quando outros fatores se somam, alguns sob seu controle e outros não, essas decisões podem contribuir para uma interrupção da produção.

> "Sua organização deve continuamente afirmar que os indivíduos NUNCA são a 'causa raiz' das interrupções."
>
> (ZWIEBACK, Dave. **Beyond Blame**: learning from failure and success in complex systems. O'Reilly Media, 2014.) (tradução livre).

Dica 4 – Fazer perguntas

Análises *post-mortem*s são um momento para agir com transparência e honestidade sobre o que aconteceu durante o incidente. Esse não é o momento para encobrir nada ou omitir pequenos detalhes.

As pessoas envolvidas

- Quais foram as circunstâncias do incidente?

- Eles estavam dormindo quando receberam o alerta?
- Como o problema os afetou emocional e fisicamente?
- O que pode ser feito para melhorar essa situação?

O processo

- A pessoa de plantão se sentiu apoiada?
- A pessoa de plantão conhecia as políticas de escalonamento da empresa e tinha acesso rápido a informações de contato para trazer outras pessoas quando necessário?
- O incidente teve relação com o conhecimento de domínio da pessoa de plantão?
- Há algo que possa ser aprimorado no processo antes do próximo incidente?

O pós-incidente

- Quanto tempo a pessoa de plantão levou para se recuperar do incidente?
- Quais medidas foram adotadas imediatamente após o incidente?

- É necessário comunicar a interrupção aos clientes?

A cultura

- O sistema de alerta esta´ funcionando?
- Você precisa fazer alterações nos horários de plantão de sua equipe?
- Como promover uma cultura mais aberta ao aprendizado com os erros?

A próxima vez

- Você esta´ mais bem preparado para a próxima interrupção?
- Que medidas podem ser adotadas para que você esteja mais bem preparado?
- A cópia do relatório *post-mortem* esta´ facilmente disponível para qualquer pessoa?

Dica 5 – Evitar punição

Se as pessoas acreditarem que serão punidas por suas ações durante uma interrupção, não se sentirão incentivadas a fornecer todos os detalhes. Elas não contam toda a verdade e ocultam informações que possam comprometê-las.

O medo de retaliação contra um funcionário após um incidente praticamente garante a repetição do problema e impede que a equipe entenda por que determinadas decisões pareciam fazer sentido no context do incidente.

É fundamental criar um senso de segurança e responsabilidade. Erros serão cometidos, mas a pessoa que cometer o erro deve ser capaz de aprender com ele, e não ser punido por isso.

_*Framework* – Definir o contexto

- Falha é uma parte normal do funcionamento de sistemas complexos. Todos os sistemas falham; é apenas uma questão de tempo.
- O objetivo do *post-mortem* é aprender. Ninguém será culpado, envergonhado ou punido por relatar o que aconteceu. Uma cultura sem culpabilização não significa ausência de responsabilidade. Na verdade, os *post-mortems* sem culpabilização são a melhor forma de reunir informações completas sobre o que aconteceu.
- Lembre aos participantes que, durante o período *post-mortem,* todos estão sob a influência de vieses cognitivos.

Os mais comuns são preconceitos de retrospectiva, resultado e recenticidade; e erro fundamentalde atribuição. Como nem sempre percebemos quando somos influenciados por esses vieses, incentive os participantes a identificá-los e mencioná-los durante o período *post-mortem*.

Framework – Construir uma linha do tempo

- Queremos entender o que aconteceu sob a perspectiva das pessoas envolvidas: o que elas sabiam, em que momento e por que isso fazia sentido?
- Descreva o que aconteceu, não explique.
- Quanto mais pontos de vista que você pode reunir, mais completa será a compreensão do incidente. Incentive opiniões divergentes e observe diferentes perspectivas.
- Como facilitador, seu trabalho é “ouvir, descobrir e verificar sintetizando”.

_*Framework* – Construir uma linha do tempo

- Queremos entender o que aconteceu sob a perspectiva das pessoas envolvidas: o que elas sabiam, em que momento e por que isso fazia sentido?
- Descreva o que aconteceu, não explique.
- Quanto mais pontos de vista que você pode reunir, mais completa será a compreensão do incidente. Incentive opiniões divergentes e observe diferentes perspectivas.
- Como facilitador, seu trabalho é "ouvir, descobrir e verificar sintetizando".

_*Framework* – Determinar, priorizar e publicar

- Determinar e priorizar os itens a serem remediados.
- Publicar o documento do *post-mortem* o mais aberto possível.

_*Checklist* básico

- Documentar sua linha do tempo ou dados de *log*.

- Documentar as conversas.
- Deixar espaço para anotações.
- Registrar métricas métricas como tempo médio de resolucão/tempos por nível de gravidade e outros indicadores de desempenho.
- Arquivar o reletório para consulta de histórico de remediação, torná-lo acionável.